Guía de estudio: sistema de gestión de importaciones

Architecture | Data flow | Evaluation preparation

Esta guía explica el proyecto implementado, cómo seguir una petición por todas sus capas y cómo practicar cambios graduales antes de una evaluación. La mejor forma de aprenderlo es abrir el código, seguir una solicitud de principio a fin, modificar una sola cosa y comprobarla con una prueba.

PHP MVC Interfaz y solicitudes HTTP	**Spring Boot** Cat?logo e Importaciones	**Eureka + Gateway** Descubrimiento y enrutamiento	**MySQL** Persistencia separada

Manual de referencia para comprender el sistema, explicar sus decisiones y practicar cambios de forma segura.

Basado en el codigo fuente del proyecto integrador

Contenido

1. Vista general 4

Puertos locales 4

2. Cómo circulan los datos 4

2.1 Al abrir la página 4

2.2 Al importar un CSV 5

2.3 Formato y reglas del CSV 5

2.4 Estados que puedes ver 5

3. Qué hace cada parte del código 5

PHP y la interfaz 5

API Gateway y Eureka 6

Servicios Spring Boot 6

Catálogo 6

Importaciones 6

4. Endpoints para probar 6

5. Arranque y diagnóstico 7

Diagnóstico rápido 8

6. Ejercicios graduales para preparar la evaluación 8

Ejercicio 1 — Cambiar el estilo del encabezado 8

Ejercicio 2 — Añadir un filtro al catálogo 8

Ejercicio 3 — Cambiar una regla de validación 8

Ejercicio 4 — Agregar un endpoint de consulta 8

Ejercicio 5 — Trazar y manejar un error 9

Ejercicio 6 — Preparar un cambio de datos 9

7. Práctica de evaluación: crear un microservicio desde cero 9

A. Definir el contrato primero 9

B. Generar el esqueleto 9

C. Construir en capas 9

D. Configurar MySQL y Eureka 10

E. Integrarlo con Gateway 10

F. Probar y explicar 10

8. Preguntas típicas del ingeniero 10

9. Método de trabajo recomendado 11

Esta guía explica el proyecto implementado, cómo seguir una petición por todas sus capas y cómo practicar cambios graduales antes de una evaluación. La mejor forma de aprenderlo es abrir el código, seguir una solicitud de principio a fin, modificar una sola cosa y comprobarla con una prueba.

1. Vista general

El sistema usa dos estilos juntos:

? **Monolito modular pequeño en PHP:** el sitio web tiene un controlador, un modelo que llama a servicios y una vista.

? **Microservicios en Spring Boot:** Catálogo e Importaciones son aplicaciones independientes, con procesos, puertos y bases de datos separados.

? **Servicios de infraestructura:** Eureka mantiene el registro de los servicios y API Gateway es la entrada común a las APIs.

? **Persistencia:** MySQL/MariaDB guarda catálogo e historial; cada servicio de negocio tiene su propia base.

Puertos locales

Componente	Puerto	Responsabilidad
Apache/XAMPP	80	Servir las páginas PHP
API Gateway	8080	Enrutar las solicitudes a los servicios
Catálogo	8081	Buscar productos, consultar categorías y procesar filas
Importaciones	8082	Aceptar CSV, registrar el proceso y exponer su historial
Eureka	8761	Registrar y descubrir instancias Spring
MySQL/MariaDB	3306	Persistir los datos

El puerto donde se ejecuta una aplicación no es lo mismo que su nombre registrado. Por ejemplo, Catálogo escucha en `8081`, pero anuncia `catalogo-service` a Eureka.

2. Cómo circulan los datos

2.1 Al abrir la página

1. El navegador pide `http://localhost/Proyecto_final/`.
2. Apache ejecuta `Proyecto_final/index.php`.
3. `index.php` crea `ImportacionController` y llama a `handleRequest()`.
4. El controlador interpreta `seccion`, filtros y parámetros de la URL.
5. Si necesita datos, `ImportacionModel` llama al API Gateway por cURL.
6. El modelo decodifica JSON y conserva el código HTTP para que la vista pueda mostrar los errores.
7. El controlador incluye `views/importacion.php`.
8. La vista genera el HTML y escapa los valores que muestra con `htmlspecialchars`.

Ejemplo: `?seccion=productos&query=mouse&categoria=Perifericos&page=0` termina en `GET /api/v1/productos?query=mouse&categoria=Perifericos&page=0&size=10`.

2.2 Al importar un CSV

1. La vista envía un formulario `POST` con `multipart/form-data`; el archivo viaja en el campo `archivo`.
2. PHP valida que se haya recibido el archivo sin error y que su extensión sea `.csv`.
3. `ImportacionModel::enviarArchivoCSV()` crea un `CURLFile` y lo envía a `POST http://localhost:8080/api/v1/importaciones/cargar`.
4. Gateway busca la instancia `IMPORTACIONES-SERVICE` registrada en Eureka y reenvía la solicitud multipart.
5. `ImportacionController.cargarCsv()` entrega el archivo al servicio.
6. `ImportacionService` crea un registro de importación en estado `PROCESANDO`, lee el CSV UTF-8 y valida encabezado, columnas y valores.
7. Las filas mal formadas se agregan al historial de errores; las filas sintácticamente válidas se reúnen en una lista.
8. Importaciones envía esa lista como JSON a `POST /api/v1/productos/lote` del Catálogo. La URL usa el nombre DNS lógico `catalogo-service`, resuelto mediante balanceo de Spring Cloud/Eureka, no una IP fija.
9. Catálogo valida cada fila contra reglas de negocio y MySQL. Responde con el resultado individual de cada fila.
10. Importaciones combina errores del CSV y resultados del Catálogo, actualiza resumen y errores en `db_importaciones`, y devuelve un DTO.
11. PHP muestra el resumen y permite abrir el detalle.

El archivo se analiza **por filas**: que una fila falle no impide evaluar las demás. Si el catálogo está desconectado, el historial guarda el fallo de comunicación en lugar de marcar la carga como exitosa.

2.3 Formato y reglas del CSV

```
codigo,nombre,categoria,precio,existencia
REP-101,"Mouse, inalámbrico",Perifericos,125.50,15
```

? UTF-8 con una fila de encabezados en este orden exacto.

? Deben existir código, nombre y categoría.

? Código: hasta 50 caracteres y único en el catálogo.

? Nombre: hasta 150 caracteres.

? La categoría debe existir; el esquema inicial registra `Perifericos` y `Monitores`.

? Precio mayor que cero; existencia entera no negativa.

? Los campos con comas pueden entrecomillarse; para incluir comillas dobles en un campo entrecomillado se duplican (`""`).

? Cada registro ocupa una línea física; no se admiten campos que continúen en otra línea.

2.4 Estados que puedes ver

? `PROCESANDO`: estado inicial mientras comienza el servicio de importación.

? `COMPLETADO`: filas procesadas sin rechazos.

? `COMPLETADO_CON_ERRORES`: algunas filas quedaron para revisión.

? `FALLIDO_VALIDACION`: encabezado vacío/incorrecto o CSV sin filas de productos.

? `FALLIDO`: no se pudo leer el archivo.

? `ERROR_COMUNICACION_CATALOGO`: el catálogo no respondió correctamente.

3. Qué hace cada parte del código

PHP y la interfaz

? `Proyecto_final/index.php`: punto de entrada, construye el controlador.

? `Proyecto_final/controllers/importacionController.php`: selecciona la página, valida parámetros y upload, y solicita datos al modelo.

- ? `Proyecto_final/models/importacionModel.php`: centraliza HTTP/cURL hacia `localhost:8080`, arma query strings, envía `CURLFile` y traduce problemas de red/JSON.
- ? `Proyecto_final/views/importacion.php`: renderiza inicio, carga, catálogo, producto, historial y detalle.
- ? `Proyecto_final/style.css` y `Proyecto_final/meso_logo.png`: estilo verde, encabezado y logo.
- ? `Proyecto_final/ejemplos/`: CSV válido y CSV con errores para practicar.

En la vista, `$h(...)` escapa texto antes de insertarlo en HTML. Conserva este patrón si agregas datos dinámicos.

API Gateway y Eureka

- ? `eureka-service`: arranca el servidor con `@EnableEurekaServer`; servicios de negocio y Gateway se registran como clientes.
- ? `api-gateway/src/main/resources/application.properties`: escucha en 8080. Sus rutas explícitas envían `/api/v1/productos/**` a `lb://CATALOGO-SERVICE` y `/api/v1/importaciones/**` a `lb://IMPORTACIONES-SERVICE`.
- ? `lb://` significa que Spring Cloud obtiene instancias disponibles de Eureka y las balancea. Si Eureka no tiene instancias activas, la ruta no puede completar la solicitud.
- ? Los servicios de negocio registran sus nombres configurados en `spring.application.name`.

Servicios Spring Boot

En cada servicio verás paquetes similares:

- ? `controller`: frontera HTTP. Traduce rutas/parámetros a llamadas de servicio y selecciona códigos/respuestas HTTP.
- ? `service`: reglas de negocio y coordinación.
- ? `repository`: interfaces Spring Data JPA; extienden `JpaRepository` para operaciones con base de datos.
- ? `model`: entidades persistentes (`@Entity`).
- ? `dto`: objetos de transferencia expuestos por el API; delimitan la forma JSON.
- ? `resources/application.properties`: puerto, nombre de Eureka y conexión local a MySQL.
- ? `src/test`: pruebas; los tests de servicio usan Mockito para no insertar datos de prueba en MySQL.

Catálogo

`ProductoController` expone `/api/v1/productos`; `ProductoService` coordina validaciones y repositorios; `iProductoRepository.buscar(...)` filtra texto en nombre/código y categoría, con `Pageable`. `Producto` se relaciona con `Categoria` por `ManyToOne`.

`POST /api/v1/productos/lote` recibe una lista de `ProductoRequestDto`. Devuelve un resultado por fila, incluso si hay duplicado o una regla de negocio falla. La API usa JSON, no recibe el archivo CSV.

Importaciones

`ImportacionController` expone cuatro operaciones: página de importaciones, detalle, errores y carga CSV. `ImportacionService` es responsable del análisis del archivo, la llamada HTTP al Catálogo, el resumen y los errores.

La base `db_importaciones` conserva `importaciones` y `importacion_errores`; la base `db_catalogo` guarda `productos` y `categorias`. La separación evita que Importaciones escriba directamente las tablas del Catálogo: coopera con el otro servicio por su API REST.

4. Endpoints para probar

Puedes usar el navegador para GET y Swagger/Postman/curl para POST. Las solicitudes regulares de la aplicación usan Gateway en `http://localhost:8080`.

Método	Ruta	Qué demuestra
GET	`/api/v1/productos?query=mouse&categoria=Perifericos&page=0&size=10`	Búsqueda paginada
GET	`/api/v1/productos/{id}`	Detalle de un producto
GET	`/api/v1/productos/categorias`	Categorías del Catálogo
POST	`/api/v1/productos/lote`	Validar/registrar productos en lote
GET	`/api/v1/importaciones?page=0&size=10`	Historial paginado
GET	`/api/v1/importaciones/{id}`	Resumen con errores del proceso
GET	`/api/v1/importaciones/{id}/errores`	Errores de ese proceso
POST	`/api/v1/importaciones/cargar`	Subir multipart con campo `archivo`

Las interfaces Swagger de los servicios están en `http://localhost:8081/swagger-ui.html` y `http://localhost:8082/swagger-ui.html`. Gateway sirve el tráfico del sistema; la documentación Swagger está configurada directamente en los dos servicios.

Ejemplo de `POST /api/v1/productos/lote`:

```
[
  {
    "numeroFila": 2,
    "codigo": "PRACTICA-001",
    "nombre": "Mouse de prueba",
    "categoria": "Perifericos",
    "precio": 125.50,
    "existencia": 15
  }
]
```

5. Arranque y diagnóstico

1. En XAMPP enciende Apache y MySQL.
2. Importa una vez `Proyecto_Final_IV/database/schema.sql` en phpMyAdmin. Si las bases ya están creadas, no borres tablas con datos.
3. Asegúrate de tener Java 17 y abre cuatro terminales PowerShell.
4. Ejecuta `.\mvnw.cmd spring-boot:run` desde cada directorio en este orden:

1. `eureka-service`
2. `Proyecto_Final_IV` (Catálogo)
3. `importaciones-service`
4. `api-gateway`

5. Comprueba Eureka en `http://localhost:8761/`. Deben aparecer Catálogo, Importaciones y Gateway.

6. Abre `http://localhost/Proyecto_final/`.

Para cada servicio puedes ejecutar pruebas con `.\mvnw.cmd test` desde su carpeta. Los tests unitarios actuales de Catálogo e Importaciones reemplazan repositorios/HTTP con mocks; no cargan CSV ni escriben datos funcionales en la base activa.

Diagnóstico rápido

? **La página no abre:** comprueba Apache y que el proyecto esté en `C:\xampp\htdocs\Proyecto_final`.

? **Página visible pero API falla:** prueba Gateway en 8080 y comprueba la lista de instancias en Eureka.

? **Gateway responde 5xx sin servicio:** confirma que los servicios aparecen `UP` y que coinciden sus nombres (`spring.application.name` y destinos `lb://...`).

? **Servicio no arranca:** revisa consola, versión Java y conflicto de puerto.

? **Error de base de datos:** comprueba que MySQL escucha en 3306, que existen las bases, y que credenciales/URL del servicio sean correctas.

? **CSV rechazado:** inspecciona el mensaje en detalle; separa errores de formato local (fila/columnas/número) de errores de negocio del Catálogo (duplicado/categoría).

? **Cambiaste código pero el navegador no:** confirma que modificaste la copia publicada por Apache y recarga; si hay más de una carpeta del proyecto, compara las rutas.

6. Ejercicios graduales para preparar la evaluación

Hazlos en una rama de práctica o guarda una copia antes. Cambia una cosa por ejercicio, ejecuta las pruebas relevantes y explica el recorrido en voz alta. No uses códigos ya existentes si pruebas contra la base real.

Ejercicio 1 — Cambiar el estilo del encabezado

1. Modifica una variable de color en `Proyecto_final/style.css`.
2. Recarga la página y observa qué partes cambian.
3. Cambia el tamaño del logo, manteniendo proporción y `border-radius: 50%`.
4. Verifica una pantalla estrecha; la regla `@media` reorganiza encabezado y navegación.

Comprueba: la página sigue cargando y el menú no desborda en móvil.

Ejercicio 2 — Añadir un filtro al catálogo

1. Lee el `<form>` de la sección de productos en la vista PHP.
2. Agrega un parámetro GET con un nombre claro y persiste el valor en el formulario.
3. Pásalo desde `ImportacionController` a `ImportacionModel::obtener`.
4. Añade el parámetro correspondiente a `ProductoController`.
5. Implementa el filtro en `iProductoRepository` y conéctalo desde `ProductoService`.
6. Actualiza `ProductoServiceTest` con un caso de coincidencia y uno sin coincidencia.

Comprueba: la URL transporta el filtro, la respuesta queda paginada y una cadena vacía conserva el comportamiento actual.

Ejercicio 3 — Cambiar una regla de validación

1. Elige una regla explícita, como el largo máximo del código.
2. Localiza la validación del parser en `ImportacionService` y la validación de negocio en `ProductoService`.
3. Decide cuál es el límite autoritativo y mantén mensajes/DTOs coherentes.
4. Agrega pruebas para un valor justo en el límite y uno que lo exceda.
5. No escribas la fila inválida en MySQL.

Comprueba: el caso permitido pasa, el excedido se rechaza y el CSV puede continuar con otras filas.

Ejercicio 4 — Agregar un endpoint de consulta

1. Diseña método y ruta HTTP (por ejemplo, un conteo o resumen del Catálogo).
2. Añade el método en `ProductoController`; deja la consulta/regla en `ProductoService`.

3. Reutiliza el repositorio si ya contiene la operación.
4. Prueba la capa de servicio con Mockito y prueba la ruta con Swagger.
5. Si la ruta sigue bajo `/api/v1/productos/**`, Gateway ya la enruta; confirma esta suposición antes de crear una ruta nueva.

Comprueba: el status, el JSON y el caso de no encontrado son intencionales.

Ejercicio 5 — Trazar y manejar un error

1. Detén solo el Catálogo o apunta temporalmente la URL del servicio a una dirección inexistente en una copia de práctica.
2. Sigue el `POST` desde `ImportacionService` y localiza cómo se registra `ERROR_COMUNICACION_CATALOGO`.
3. Comprueba que historial y detalle muestran la causa general sin exponer stack traces al usuario.
4. Restaura la configuración y verifica el estado `UP` en Eureka.

Comprueba: la importación no queda con estado `COMPLETADO` cuando el catálogo no pudo confirmar los resultados.

Ejercicio 6 — Preparar un cambio de datos

1. Dibuja las tablas y relaciones afectadas antes de editar el código.
2. Agrega una columna a la entidad y al esquema SQL (o deja que `ddl-auto=update` la cree solo en una base desechable).
3. Decide qué DTO y pantallas/API deben exponer el nuevo campo.
4. Actualiza fixtures y pruebas.
5. Para una entrega real, crea una migración SQL repetible y prueba con datos de respaldo; no dependas de `update` en producción.

Comprueba: entity, JSON, base y vista concuerdan y los registros anteriores siguen legibles.

7. Práctica de evaluación: crear un microservicio desde cero

Practica fuera de los servicios existentes, por ejemplo `inventario-service`, para no poner en riesgo el Catálogo actual. La meta es demostrar un CRUD pequeño; evita arrancar con muchas reglas.

A. Definir el contrato primero

Propón una entidad `Almacen` con `id`, `nombre` y `ubicacion`. Escribe antes las rutas:

```
GET    /api/v1/almacenes
GET    /api/v1/almacenes/{id}
POST   /api/v1/almacenes
PUT    /api/v1/almacenes/{id}
DELETE /api/v1/almacenes/{id}
```

Decide respuestas: colección `200`, detalle `200`/`404`, creación `201`, actualización `200`/`404`, borrado `204`/`404`, entrada inválida `400`. No devuelvas `200` para una operación fallida.

B. Generar el esqueleto

1. Usa Spring Initializr o copia el esqueleto Maven conocido, con Java 17 y una versión compatible con Spring Boot 3.2.5 / Spring Cloud 2023.0.1.
2. Agrega dependencias equivalentes a Catálogo: Spring Web, Spring Data JPA, MySQL Connector, Eureka Client y Spring Boot Test.
3. Usa el Maven Wrapper si el nuevo directorio lo incluye.
4. Arranca la aplicación vacía; confirma que compila antes de agregar persistencia.

C. Construir en capas

1. Crea `InventarioServiceApplication` con `@SpringBootApplication` y `main`.
2. Crea `model/Almacen` con `@Entity`, `@Id`, `@GeneratedValue` y `@Column` según las reglas.
3. Crea `repository/AlmacenRepository extends JpaRepository<Almacen, Long>`.

4. Crea `service/AlmacenService` con inyección por constructor y las operaciones del contrato.
5. Crea `controller/AlmacenController` con `@RestController`, `@RequestMapping("/api/v1/almacenes")`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` y `@PathVariable`.
6. Usa DTOs para las solicitudes/respuestas si el contrato no debe exponer directamente la entidad; valida entradas con Jakarta Validation si se agregan las dependencias/anotaciones correspondientes.
7. Maneja IDs inexistentes de forma explícita con `Optional` y `ResponseEntity.notFound()`.

No escribas lógica de negocio en el controlador ni SQL manual dentro de la vista. Mantén cada clase en su capa.

D. Configurar MySQL y Eureka

Crea una base aislada, por ejemplo `db_inventario`. En `application.properties` configura el nombre y un puerto disponible:

```
server.port=8083
spring.application.name=inventario-service
spring.datasource.url=jdbc:mysql://localhost:3306/db_inventario|useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.instance.hostname=localhost
```

Usa el usuario/contraseña que realmente tengas. En una instalación compartida, no confirmes contraseñas en Git. `ddl-auto=update` es conveniente para la práctica local; en un sistema serio usa migraciones explícitas.

E. Integrarlo con Gateway

Agrega una ruta explícita a `api-gateway/src/main/resources/application.properties`:

```
spring.cloud.gateway.routes[2].id=inventario-route
spring.cloud.gateway.routes[2].uri=lb://INVENTARIO-SERVICE
spring.cloud.gateway.routes[2].predicates[0]=Path=/api/v1/almacenes/**
```

Spring Cloud Gateway normalmente normaliza el ID para resolver el nombre de servicio en Eureka. Si la ruta no funciona, valida el nombre real en Eureka, el ID configurado y la ruta registrada; no adivines.

F. Probar y explicar

1. Inicia MySQL, Eureka, `inventario-service` y Gateway.
2. Verifica que `INVENTARIO-SERVICE` aparezca activo en `http://localhost:8761/`.
3. Crea un almacén mediante `POST` en Gateway con JSON; consulta colección/detalle.
4. Prueba un ID inexistente y un cuerpo inválido.
5. Añade tests unitarios del servicio usando repositorio mock; prueba al menos alta, búsqueda y not found.
6. Ejecuta `.\mvnw.cmd test` antes de levantar el servicio junto al clúster: algunas pruebas de contexto levantan instancias que pueden registrarse temporalmente en Eureka.
7. Explica sin leer: ruta → Controller → Service → Repository → MySQL, y de vuelta la respuesta; luego explica cómo Gateway encuentra la instancia.

Rúbrica de autoevaluación: compila; tiene entidad/repositorio/servicio/controller separados; persiste en una BD aislada; se registra en Eureka; se accede por Gateway; responde correctamente 200/201/204/400/404; cuenta con pruebas y README/configuración verificables.

8. Preguntas típicas del ingeniero

¿Por qué hay Gateway y Eureka? Gateway expone una entrada estable y enruta; Eureka mantiene los nombres y ubicaciones de las instancias disponibles. No son lo mismo.

¿Por qué Importaciones no inserta en XINLINE0TOKENX directamente? La base del Catálogo pertenece a Catálogo. Importaciones pide registrar filas a través de su API para conservar reglas y límites del servicio.

¿Qué diferencia hay entre Controller, Service y Repository? HTTP/contrato; reglas/orquestación; persistencia, respectivamente.

¿Por qué existen DTOs? Para definir un contrato de red separado de las entidades y enviar solo los datos requeridos.

¿Qué diferencia hay entre error local y error de negocio? Formato inválido se rechaza al leer el CSV; duplicado/categoría inexistente se decide en Catálogo con el estado real de la base.

¿Qué ocurre si una fila está mal? Se registra con número y motivo; otras filas válidas siguen su flujo.

¿Cómo empiezo un cambio frente al evaluador? Reformula requisito, identifica endpoint y flujo, encuentra las capas tocadas, implementa en pasos pequeños, agrega una prueba y ejecuta el test antes de modificar otra cosa.

9. Método de trabajo recomendado

Para cada cambio:

1. Reproduce el comportamiento actual y anota la ruta exacta.
2. Escribe qué debería cambiar y qué comportamiento no debe romperse.
3. Sigue el flujo con búsqueda de símbolos o referencias antes de editar.
4. Cambia una capa a la vez; actualiza controlador, servicio, repositorio/DTO/vista según corresponda.
5. Agrega una prueba que falle antes del arreglo y pase después.
6. Ejecuta el test focalizado; luego la compilación o suite del servicio.
7. Prueba HTTP desde el navegador/Swagger y revisa Eureka/logs si cruza servicios.
8. Revisa `git diff`, documenta configuración/migración y crea un commit que describa el cambio real.

No publiques contraseñas, archivos CSV con datos personales ni volcados de base. Haz experimentos de carga en una base local desechable o usa el repositorio mock de las pruebas.